

MACHINE LEARNING FOUNDATIONS

Learning Problems and Perspectives

Supervised, unsupervised, and reinforcement learning — viewed through symbolic and statistical perspectives, and the practical issues that shape how learning systems are built.

Supervised · Unsupervised · Reinforcement



What Is a Learning Problem?

The classical definition that frames every paradigm that follows

A program is said to learn if:

Its performance on a task improves with experience, when measured against a clear performance criterion.

T — Task

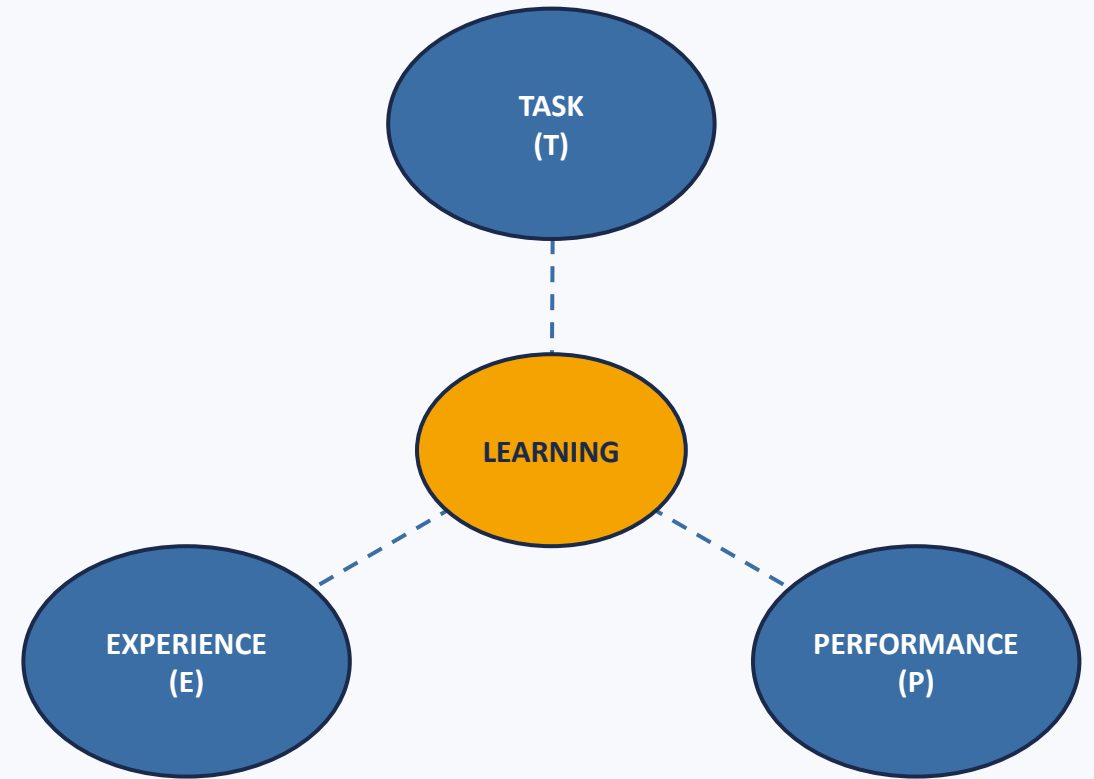
What the system is asked to do: classify emails, cluster customers, choose a move.

E — Experience

The data or interaction history the system learns from: examples, observations, trial and error.

P — Performance Measure

How success is judged: accuracy, error rate, cumulative reward, cluster quality.



Three Types of Learning Problems

Every ML task falls into one of three problem settings, defined by what feedback the learner receives



Supervised Learning

Learns a mapping from inputs to known, labeled outputs.

Spam detection, price prediction



Unsupervised Learning

Finds hidden structure in data with no labels provided.

Customer segmentation, anomaly detection



Reinforcement Learning

Learns by acting in an environment to maximize reward.

Game playing, robotic control

Supervised Learning

Learning a function from labeled examples

- **Training signal**

Each example pairs an input with the correct output (a label).

- **Goal**

Learn a mapping $f(x) \rightarrow y$ that generalizes to unseen inputs.

- **Common tasks**

Classification (discrete labels) and regression (continuous values).

- **Typical algorithms**

Linear/logistic regression, decision trees, SVMs, neural networks.



Labeled Training Examples



Unsupervised Learning

Discovering structure without labeled outcomes

- **Training signal**

Only raw, unlabeled inputs are available — no correct answers given.

- **Goal**

Uncover hidden patterns, groupings, or compact representations.

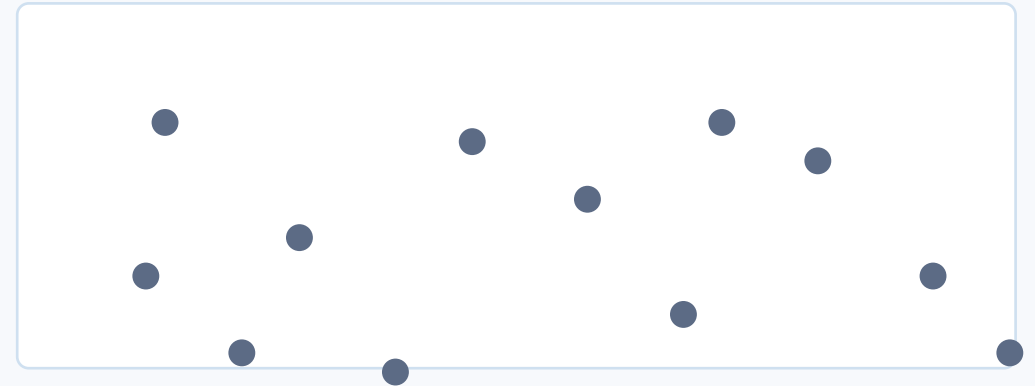
- **Common tasks**

Clustering, dimensionality reduction, association-rule mining.

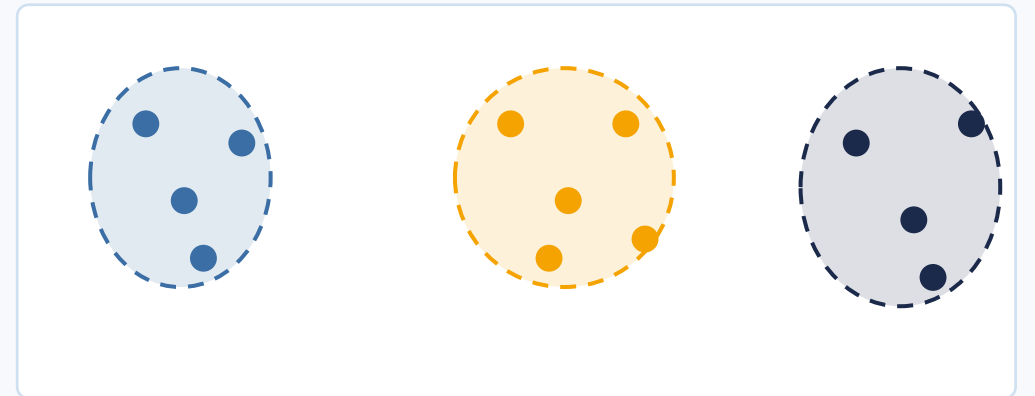
- **Typical algorithms**

k-means, hierarchical clustering, PCA, autoencoders.

Before: no structure



After: clusters revealed



Reinforcement Learning

Learning through interaction, trial, and reward

- **Training signal**

Sequential feedback in the form of rewards or penalties, not fixed labels.

- **Goal**

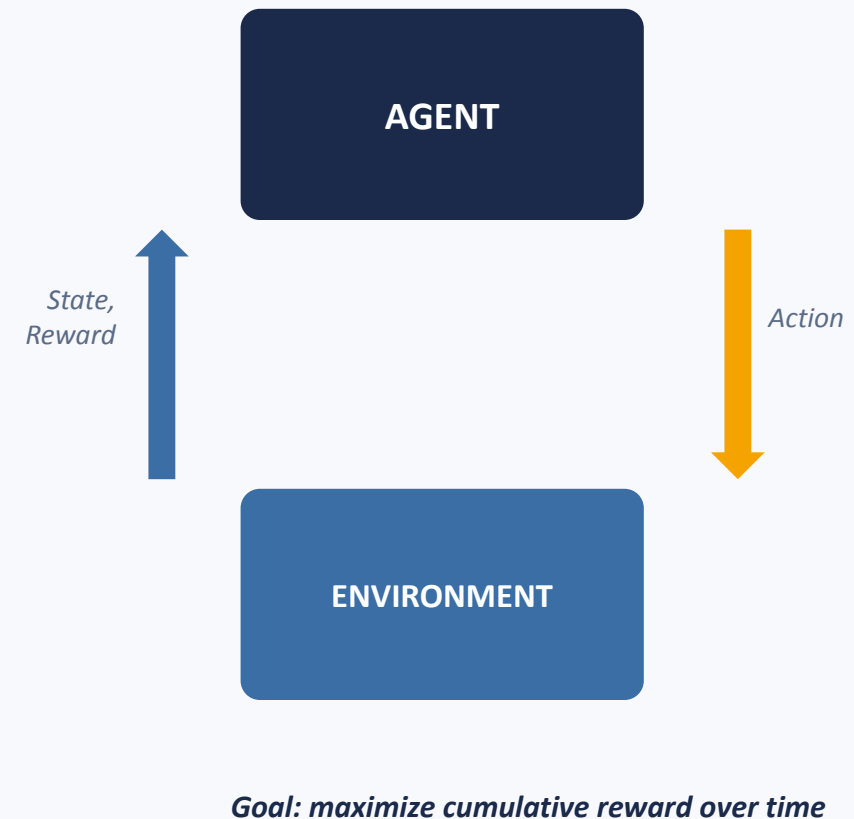
Learn a policy that maximizes long-term cumulative reward.

- **Common tasks**

Sequential decision-making under uncertainty.

- **Typical algorithms**

Q-learning, policy gradients, actor-critic, deep RL.



Comparing the Three Paradigms

Same goal — build a system that learns — with fundamentally different feedback structures

Aspect	Supervised	Unsupervised	Reinforcement
Data	Labeled input-output pairs	Unlabeled inputs only	State/action sequences, no fixed labels
Feedback	Correct answer given directly	None — structure is inferred	Delayed reward signal
Goal	Predict output for new input	Reveal hidden patterns/groups	Learn an optimal policy
Example	Email spam classification	Customer segmentation	Game-playing agents

Perspectives on Learning: Symbolic vs. Statistical

Two philosophies for building intelligent systems

Symbolic Approach

Knowledge as explicit rules

- Represents knowledge with logic, rules, and symbols
- Reasoning follows explicit inference procedures
- Highly interpretable — decisions can be traced
- Struggles with noisy, high-dimensional real-world data
- Examples: expert systems, decision rules, logic programs

Statistical Approach

Knowledge as learned patterns

- Represents knowledge as probability and weights learned from data
- Reasoning is pattern matching over numerical models
- Scales well to large, messy, real-world datasets
- Often less interpretable — a 'black box'
- Examples: regression, neural networks, probabilistic models

Key Issue: Data Quality

Learning systems are only as good as the data they learn from

1

Noise & errors

Mislabeled or inaccurate records mislead the learning process.

2

Missing values

Incomplete records force imputation or bias the sample.

3

Bias

Unrepresentative data leads to unfair or skewed predictions.

4

Insufficient volume

Too few examples limit what patterns can be reliably learned.

5

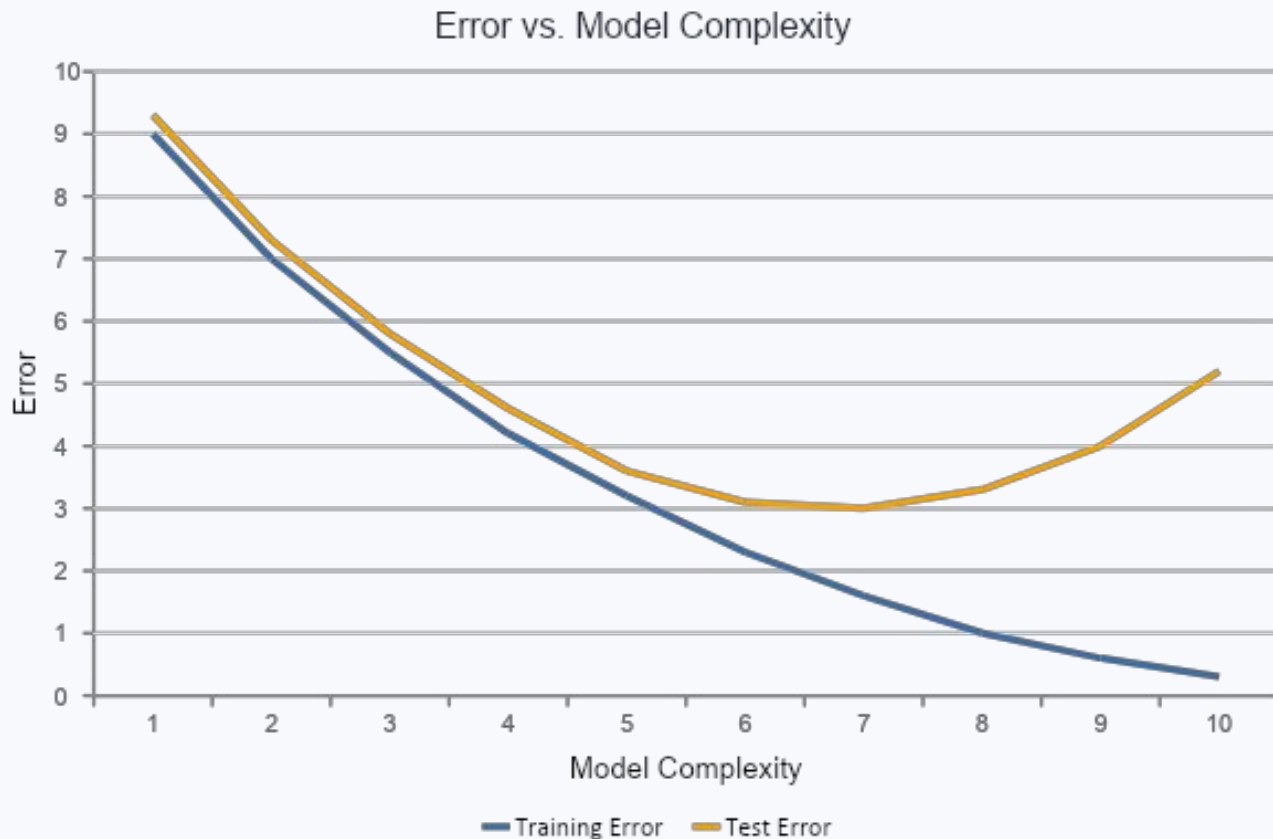
Inconsistent labeling

Different annotators or standards create contradictory signals.

Poor data quality directly caps model accuracy — no algorithm can fully compensate for it.

Key Issue: Overfitting & Underfitting

Balancing how tightly a model fits its training data



Underfitting

Model is too simple to capture the pattern — high error on both training and test data.

Overfitting

Model memorizes noise in training data — low training error, high test error.

Mitigation

Regularization, cross-validation, more data, and simpler models help find the balance.

Key Issue: Computational Complexity

The cost of training and running a learning system at scale

- **Training cost**

Larger models and datasets demand more compute time and memory.

- **Inference cost**

Real-time systems need predictions fast, constraining model choice.

- **Scalability**

Algorithms must handle growing data volume without breaking down.

- **Hardware demands**

Deep models often require GPUs/TPUs and distributed training.



Designing Effective Learning Systems

How data quality, overfitting, and computational cost shape every design choice

1

Choice of paradigm

Data quality and volume often determine whether supervised, unsupervised, or reinforcement learning is even feasible.

2

Model complexity

The overfitting/underfitting trade-off guides how expressive a model should be for the data available.

3

Deployment constraints

Computational limits decide whether a lightweight symbolic rule or a large statistical model is practical.

4

System reliability

Robust systems combine clean data pipelines, validated models, and resource-aware algorithm choices.